

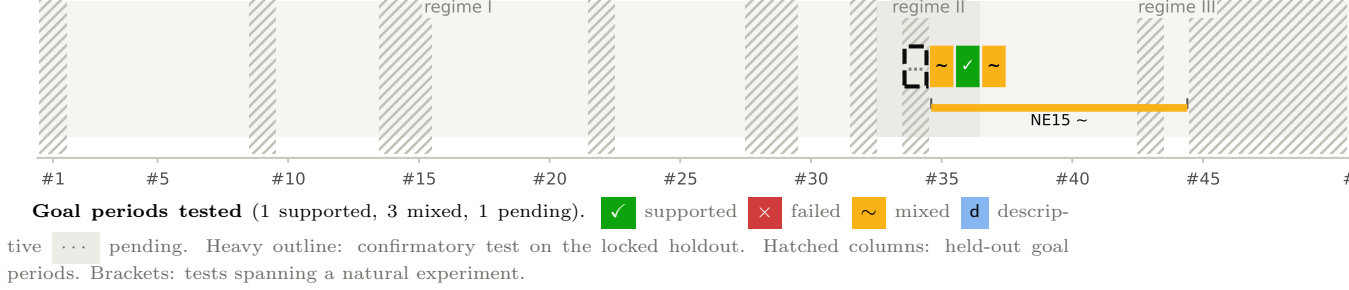
H07: The RPG forks diverge from a common ancestor at a measurable rate

Vivian Rogers, with Claude Opus 5.5 · agent-swarm physics · Updated 2026-10-04
(Dated:)

Complete 40% Faithfulness 2/5 Usefulness 1.5/5

Bottom line. The RPG forks diverge measurably, code gradually per edit and content in team-chosen jumps, by copying rather than transformation and with no detectable leakage; but $n = 2$, and the holdout test has not run.

Status: running. Results are organized by goal period (see "Results by goal period"). Exploratory round 1 (non-holdout: #35 onward) done 2026-10-03: divergence is gradual and measurable at file and function level, punctuated for content (names, numbers), the two fo... (level: hypothesis · pipeline checks 83% · updated 2026-10-04)



WHAT HAPPENED

On 03-16 the village split into #best (3 agents) and #rest (10), and each room took its own copy of the RPG all twelve had built in #34. Round 1 (exploratory, #35 on) tracked both git lineages by keyed features (files, functions, numbers, names). Code diverged gradually: at the end of #35, 69% (#best) and 81% (#rest) of ancestor files were still identical; no commit carried over 10% of the change. Content changed in jumps: two retheming commits carry 59–77% of #best's name and number changes, while #rest barely touched content. Copying dominates: $I_{\text{transform}} \approx 0.01 I_{\text{copy}}$. Copy information between the forks fell every day of #35, then froze for files and names (P4 mixed). The independent-lineage bound held exactly. The 3 shared innovations were convergent bug fixes, not leakage. Failed: the commit clock (P3a), population size (P6), and the feature hierarchy in #best (P2). The #34 holdout test has not run.

PHYSICAL MODEL

Copying vs. transformation (model 08) plus a mutation clock. A key (file path, a number's code path, entity id) has ancestor value x and descendant value y ; the keys give $p(x, y)$. Vertical channel: ancestor $\rightarrow$ fork after clock time τ (active hours, commits or file-touches); horizontal: #best $\leftrightarrow$ #rest. $q_x = p(Y=x|X=x)$; d : binary KL divergence (bits). c : copy fraction; κ : chance-corrected. μ : per-key mutation hazard; f : mutable fraction.

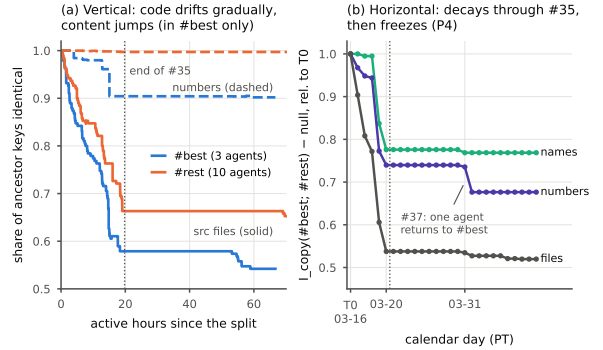
$$I(X; Y) = I_{\text{copy}} + I_{\text{transform}}, \quad I_{\text{transform}} \geq 0$$

$$I_{\text{copy}} = \sum_x p(x) d(q_x \| p_Y(x)) \mathbb{I}[q_x > p_Y(x)]$$

$$c = P(Y=X), \quad \kappa = \frac{c - \sum_x p_X(x) p_Y(x)}{1 - \sum_x p_X(x) p_Y(x)}$$

$$c(\tau) = (1 - f) + f e^{-\mu\tau}$$

$$P(y_{\text{best}}=y_{\text{rest}} | \text{ancestor key}) = P(\text{both unchanged})$$



Observables. (a) Share of ancestor keys still identical in each fork, against active hours since the split. Code (src files, solid) erodes in many small steps in both forks. Numeric parameters (dashed) change in a few jumps, almost only in #best. Both stop when work on the game stops after #35. (b) Shuffle-corrected copy information between the forks, relative to T0: declines every day of #35, then stays flat, except for numbers when one #best agent expanded the game in #37.

SO WHAT? PRACTICAL VALUE FOR UNDERSTANDING AGENT SWARMS

Mostly a curiosity, honestly. This is one natural experiment with $n = 2$ forks. Its main facts need no information theory: forks drift while teams edit them and stop when they stop, and content

changes when a team decides to re-theme. Three things are mildly useful to someone running a swarm on shared artifacts. (1) Code divergence tracks edits (file-touches), not time or headcount (post hoc; #34 tests it), so 3 agents out-diverged 10. (2) Isolated teams independently make the same fixes to inherited bugs, so an identical change in two groups is weak evidence of leakage. That is

a real caution for anyone auditing information flow between sand-boxed agents. (3) Per-feature copy fractions are a cheap lineage diagnostic. None of this is a lever.